UFRGS
UNIVERSIDADE FEDERAL DO RIO GRANDE DO SUL

Curso de Especialização

Inteligência Artificial para Engenheiros

Disciplina: **Algoritmos e Programação (APE)**

Aula: 07 - Geração de gráficos com *matplotlib*

IA PARA ENGENHEIROS
UFRGS

Professor: **Raphael Martins Brum**

2026

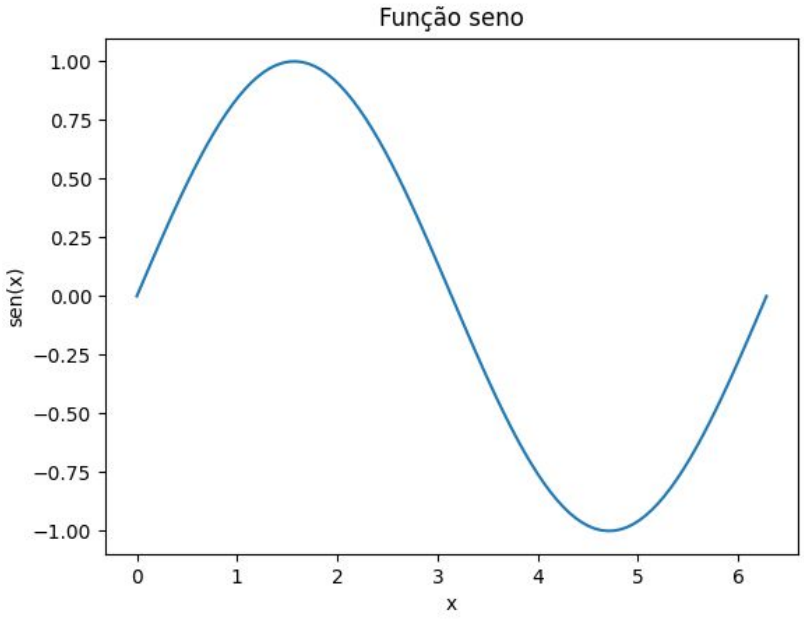
Biblioteca matplotlib

- *Matplotlib: Visualization with Python*
- Gera gráficos para visualização de diversos tipos de dados
- Trabalha com formatos vetoriais e *raster (bitmaps)*: PNG, PDF, SVG, EPS
- Renderiza sobre a interface gráfica ou para arquivos
- Integra-se com NumPy, Pandas, SciPy e outras

Criada por John Hunter em 2003

Primeiro exemplo

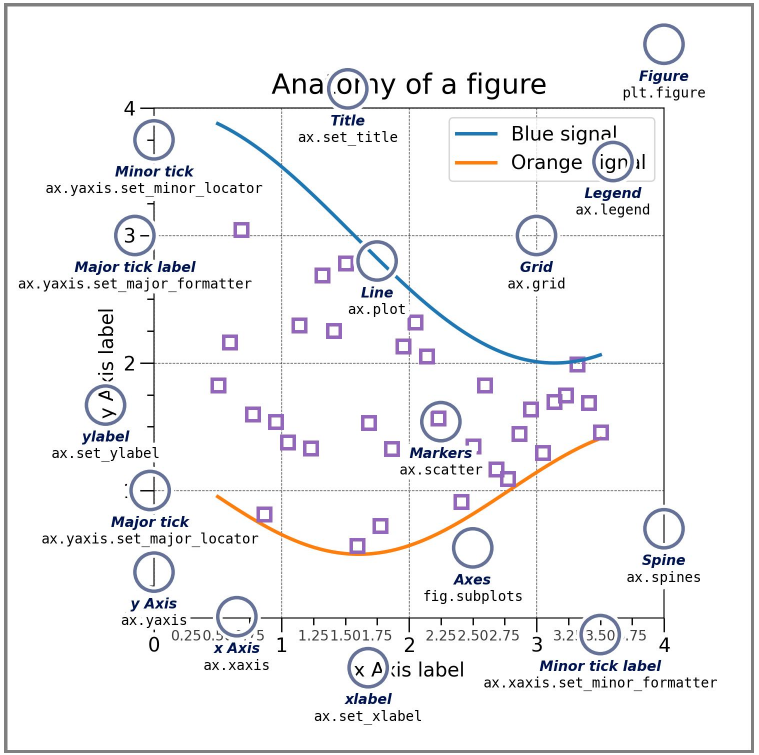
```python
import matplotlib.pyplot as plt
import numpy as np

x = np.linspace(0, 2 * np.pi,
100)
y = np.sin(x)

plt.plot(x, y)
plt.title("Função seno")
plt.xlabel("x")
plt.ylabel("sen(x)")
plt.show()
```



Partes de uma “Figura”

- Figure: canvas, pode conter um ou mais Axes
- Axes: área de plotagem com eixos, ticks e labels
- Axis: o eixo x ou y individualmente (ticks, limites)
- Artist: tudo que é desenhado (linhas, textos, patches)

```
fig, ax = plt.subplots(figsize=(8, 5))
ax.plot(x, y)
ax.set_title("Título")
ax.set_xlabel("Eixo X")
ax.set_ylabel("Eixo Y")
```



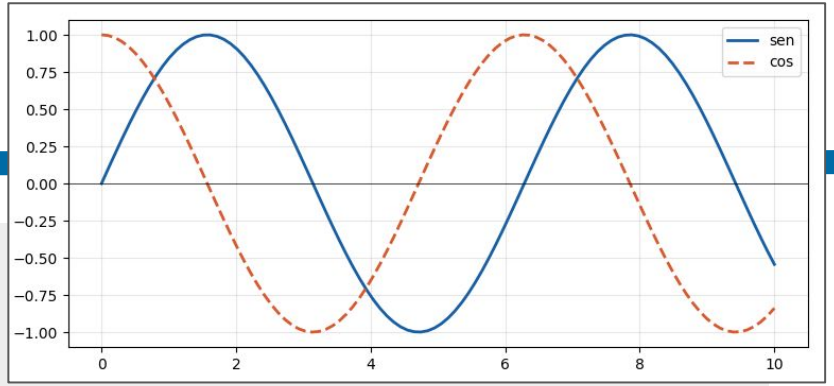
Criando o “canvas” de diversas formas

```
fig, ax = plt.subplots(figsize=(9, 4))
```

```
fig = plt.figure(figsize=(8, 5))
ax  = fig.add_subplot(111)
ax.plot(x, y)
```

```
plt.plot(x, y)
ax  = plt.gca()
fig = plt.gcf()
```

Gráfico de linha: plot()



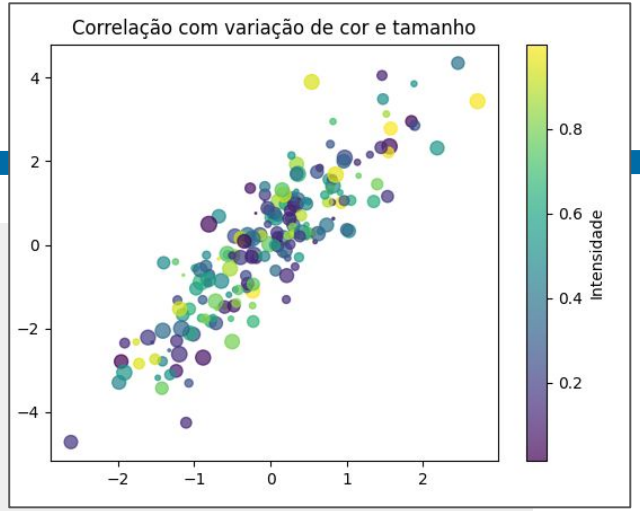

```python
fig, ax = plt.subplots(figsize=(9, 4))

x = np.linspace(0, 10, 100)

ax.plot(x, np.sin(x), label="sen",  color="#185FA5", linewidth=2)
ax.plot(x, np.cos(x), label="cos", color="#D85A30", linewidth=2, linestyle="--")

ax.legend()
ax.grid(True, alpha=0.3)
ax.axhline(y=0, color="black", linewidth=0.5)
plt.show()
```

color
Nome, hex ou RGB — ex: "red", "#185FA5", (0.2, 0.5, 0.8)

linestyle
"-", "--", "-.", ":" — ou None para pontilhado

linewidth
Espessura em pontos — padrão é 1.5

Gráfico de dispersão/xy: scatter()



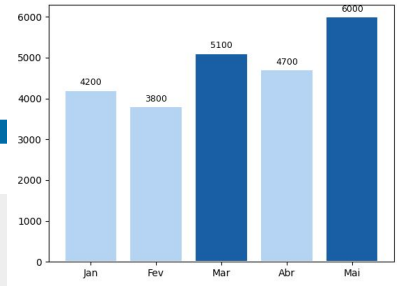

```
np.random.seed(42)
x = np.random.randn(200)
y = x * 1.5 + np.random.randn(200) * 0.8
cores = np.random.rand(200)      # cor de cada ponto
tamanhos = np.random.rand(200) * 100

fig, ax = plt.subplots()
sc = ax.scatter(x, y, c=cores, s=tamanhos, cmap="viridis", alpha=0.7)
fig.colorbar(sc, ax=ax, label="Intensidade")
ax.set_title("Correlação com variação de cor e tamanho")
```

c: cor (array ou string)
s: tamanho dos pontos

cmap: colormap: viridis, plasma, coolwarm…
alpha: transparência [0,1]

marker: "o", "^", "s", "+"
edgecolors: borda dos pontos

Gráfico de barras: bar() / barh()




```
categorias = ["Jan", "Fev", "Mar", "Abr", "Mai"]
valores    = [4200, 3800, 5100, 4700, 6000]
cores      = ["#185FA5" if v >= 5000 else "#B5D4F4" for v in valores]

fig, ax = plt.subplots()
bars = ax.bar(categorias, valores, color=cores, edgecolor="white",
linewidth=1.5)

# Anotar valor em cima de cada barra
for bar in bars:
    ax.text(bar.get_x() + bar.get_width() / 2,
            bar.get_height() + 80, str(bar.get_height()),
            ha="center", va="bottom", fontsize=9)
```

Histograma

```
dados = np.random.normal(170, 10,
500)
ax.hist(dados,
        bins=30,
        color="#378ADD",
        edgecolor="white",
        density=True)
ax.set_xlabel("Altura (cm)")
ax.set_ylabel("Densidade")
```

bins: número ou lista de bins
density=True: normaliza (para PDF)

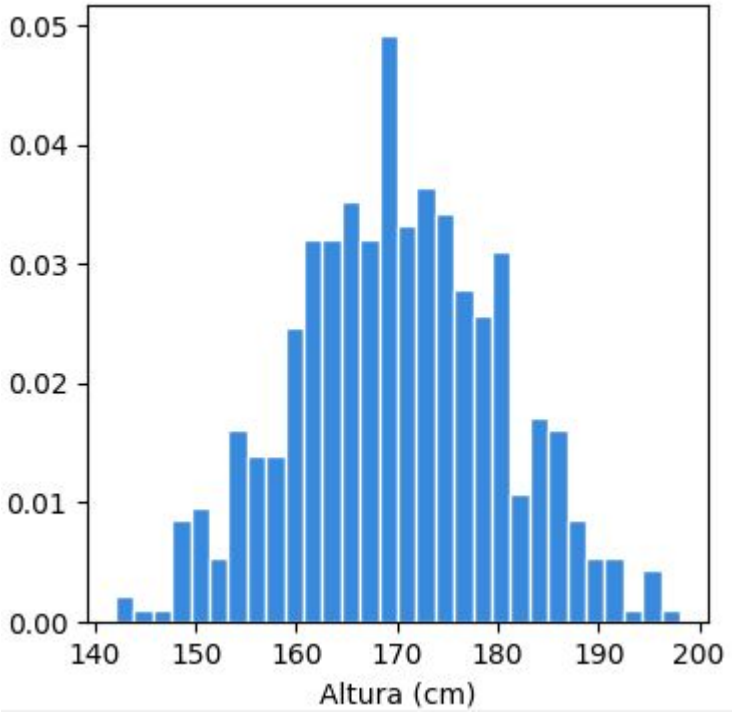
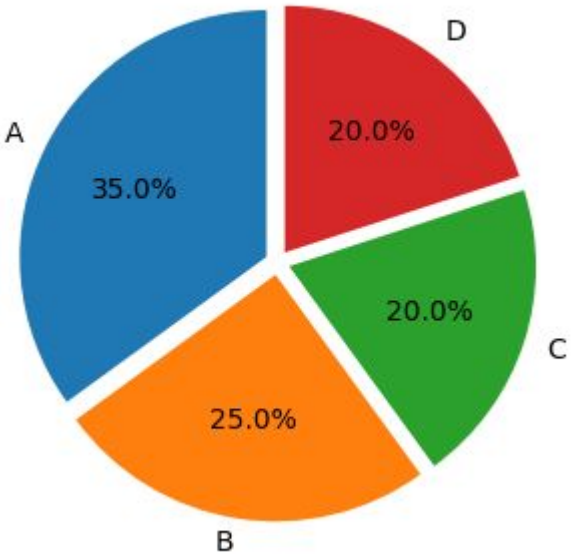


Gráfico de pizza

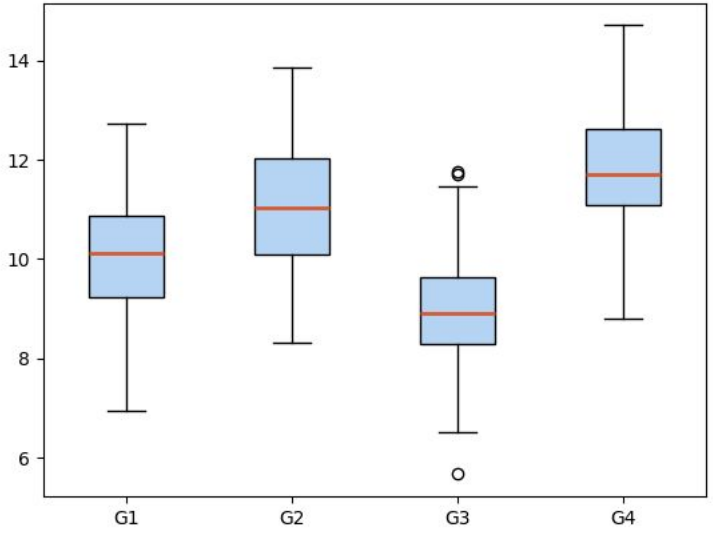



```python
fatias = [35, 25, 20, 20]
labels = ["A", "B", "C", "D"]
explode = [0.05] * 4

ax.pie(fatias,
       labels=labels,
       explode=explode,
       autopct="%1.1f%%",
       startangle=90)
```

autopct: formato dos percentuais
explode: destaca uma fatia

Boxplot: visualizando distribuições

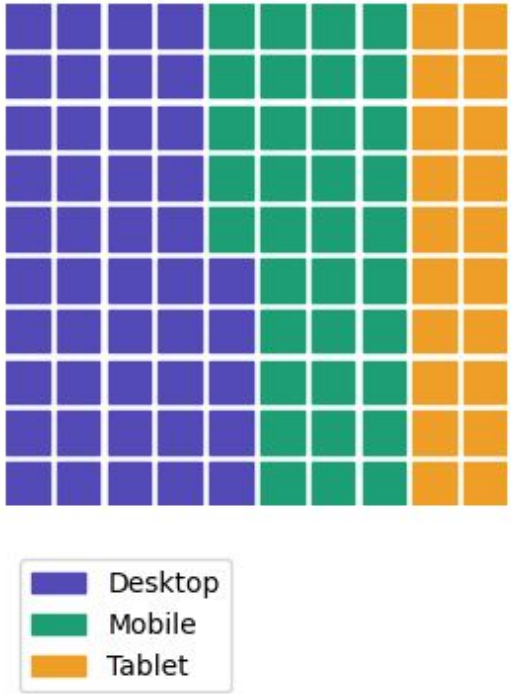
```
np.random.seed(0)
dados = [np.random.normal(m, 1.2, 100)
for m in [10, 11, 9, 12]]

fig, ax = plt.subplots()
bp = ax.boxplot(dados,
  labels=["G1", "G2", "G3", "G4"],
  patch_artist=True,
  medianprops=dict(
    color="#D85A30",
    linewidth=2))

for patch in bp["boxes"]:
    patch.set_facecolor("#B5D4F4")
```



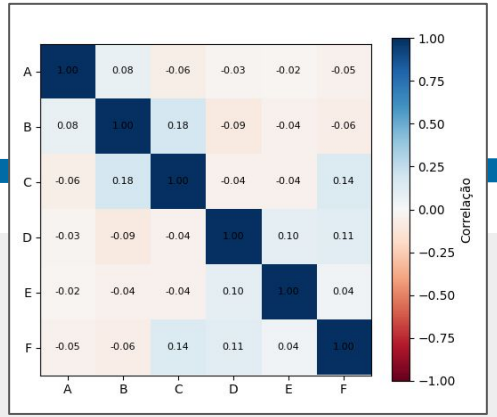
Box = Mediana + quartis + *whiskers* (1,5*IQR) + *outliers*

Waffle plot: ideal para proporções (esqueçam a pizza!)

```python
from pywaffle import Waffle
import matplotlib.pyplot as plt

dados = { "Desktop": 45,  "Mobile":  35, "Tablet":  20,}

fig = plt.figure(
    FigureClass=Waffle,
    rows=10,
    values=dados,
    colors=["#534AB7", "#1D9E75", "#EF9F27"],
    legend={
        "loc": "lower left",
        "bbox_to_anchor": (0, -0.4),
        "fontsize": 10,
    },
    figsize=(6, 4),
)
plt.show()
```



Heatmap: visualizando matrizes

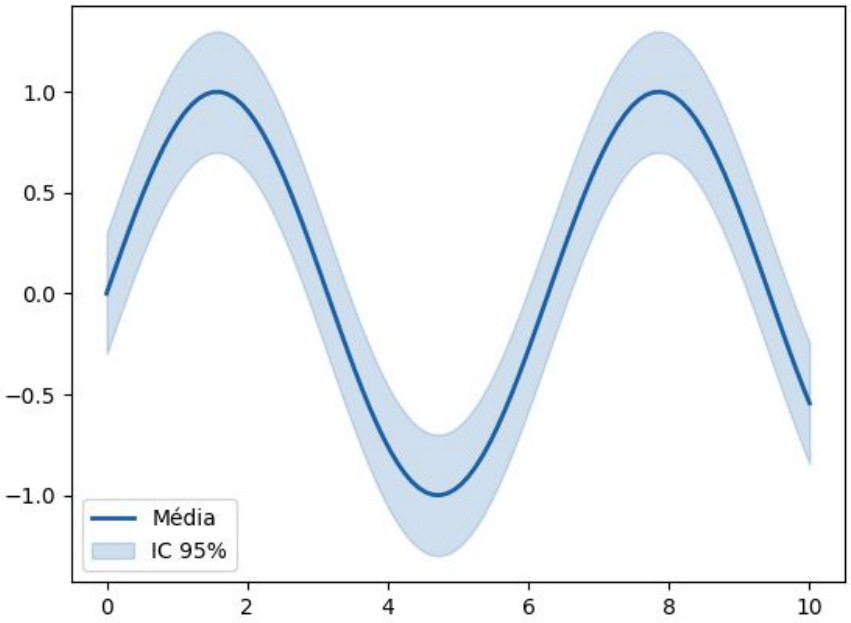



```python
np.random.seed(0)
dados = np.random.randn(6, 200)
corr = np.corrcoef(dados)

labels = ["A","B","C","D","E","F"]
fig, ax = plt.subplots(figsize=(6, 5))

im = ax.imshow(corr, cmap="RdBu",
               vmin=-1, vmax=1)
ax.set_xticks(range(6));
ax.set_xticklabels(labels)
ax.set_yticks(range(6));
ax.set_yticklabels(labels)
```

```python
fig.colorbar(im, ax=ax,
  label="Correlação")

# Anotar cada célula com o valor
for i in range(6):
    for j in range(6):
        ax.text(j, i,
          f"{corr[i,j]:.2f}",
          ha="center", va="center",
          fontsize=8, color="black")
```

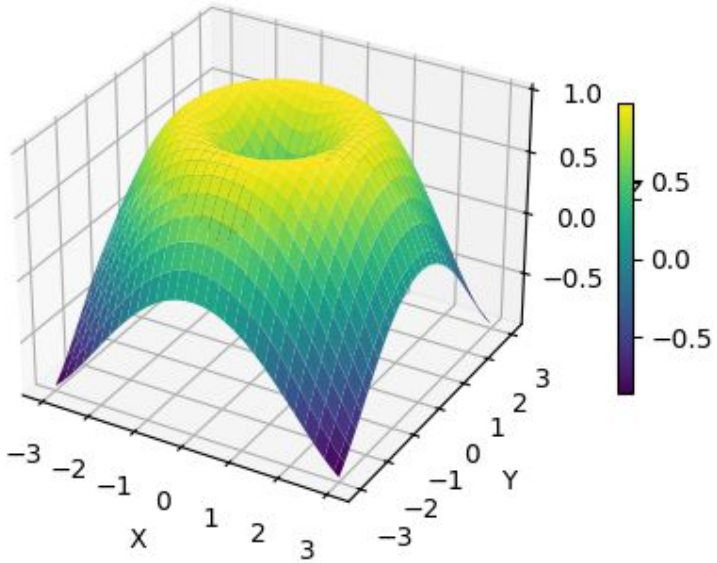
fill_between(): destacando áreas relevantes

```
x = np.linspace(0, 10, 100)
y     = np.sin(x)
y_min = y - 0.3
y_max = y + 0.3

fig, ax = plt.subplots()
ax.plot(x, y, color="#185FA5", lw=2,
       label="Média")
ax.fill_between(x, y_min, y_max,
                alpha=0.2,
                color="#185FA5",
                label="IC 95%")
ax.legend()
```



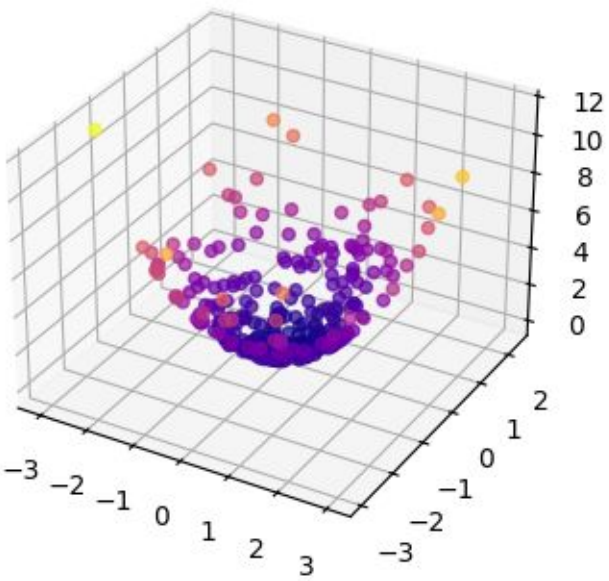
Gráficos 3D: superfícies (z = f(x,y), meshgrid)

```
from mpl_toolkits.mplot3d import Axes3D
fig = plt.figure(figsize=(8, 4))
ax  = fig.add_subplot(111, projection="3d")
x = np.linspace(-3, 3, 60)
y = np.linspace(-3, 3, 60)
X, Y = np.meshgrid(x, y)
Z = np.sin(np.sqrt(X**2 + Y**2))
surf = ax.plot_surface(X, Y, Z,
           cmap="viridis",
           linewidth=0,
           antialiased=True)
fig.colorbar(surf, shrink=0.5)
ax.set_xlabel("X")
(...)
```



Gráficos 3D: dispersão 3D (vetores x, y, z)

```
from mpl_toolkits.mplot3d import Axes3D

n = 300
xs = np.random.randn(n)
ys = np.random.randn(n)
zs = xs**2 + ys**2

ax.scatter(xs, ys, zs,
           c=zs,
           cmap="plasma",
           s=20,
           alpha=0.7)
```



Aparência: cores, estilos e fontes

```python
# Estilos pré-definidos
plt.style.use("seaborn-v0_8-whitegrid")

# Ou configure manualmente:
plt.rcParams.update({
  "font.family": "DejaVu Sans",
  "font.size": 12,
  "axes.titlesize": 14,
  "axes.spines.top": False,
  "axes.spines.right": False,
  "figure.dpi": 150,
})
```

Estilos disponíveis:

`ggplot` — fundo cinza, estilo R
`bmh` — Bayesian Methods for Hackers
`dark_background` — fundo escuro
`seaborn-v0_8-*` — várias variantes
`fivethirtyeight` — estilo editorial

`plt.style.available`

Subplots: múltiplos gráficos em um mesmo canvas

```
fig, axes = plt.subplots(nrows=2, ncols=2, figsize=(10, 7))
fig.suptitle(
  "Painel de análise",
  fontsize=14,
  fontweight="bold")

axes[0, 0].plot(x, y1)
axes[0, 1].scatter(x, y2)
axes[1, 0].bar(cats, vals)
axes[1, 1].hist(dados, bins=25)

plt.tight_layout()
plt.show()
```

Agregação de comentários e outros textos

```
fig, ax = plt.subplots()
ax.plot(x, y)
# Texto simples em coordenadas de dados
ax.text(3.14, 1.05, "Pico", ha="center", fontsize=10)
# Anotação com seta
ax.annotate("Cruzamento zero",
    xy=(3.14, 0),            # ponto indicado
    xytext=(4.5, 0.5),       # posição do texto
    arrowprops=dict(arrowstyle="->", color="gray"),
    fontsize=9)
# ax.axvline(x=val) - linha vertical em x
# ax.axhspan(y1, y2) - faixa horizontal colorida
# ax.fill_between(x, y1, y2) - área entre duas curvas
```

Integração com Pandas

```python
import pandas as pd

df = pd.DataFrame({
    "mes": ["Jan", "Fev", "Mar", "Abr", "Mai"],
    "receita": [42000, 38000, 51000, 47000, 60000],
    "custo":   [28000, 25000, 34000, 30000, 39000],
})

# pandas plot usa matplotlib por baixo
ax = df.plot(x="mes", y=["receita", "custo"],
             kind="bar", figsize=(8, 4))
ax.set_title("Receita vs. Custo")
fig = ax.get_figure()   # recuperar o Figure para salvar
```

Exportando figuras com savefig()

```python
# Alta resolução para publicação
fig.savefig("grafico.png", dpi=300, bbox_inches="tight")
# Vetor para impressão
fig.savefig("grafico.pdf", bbox_inches="tight")
# Web - fundo transparente
fig.savefig("grafico.svg", transparent=True)
```

```python
# Em memória (para web apps, APIs)
from io import BytesIO
buf = BytesIO()
fig.savefig(buf, format="png", dpi=150)
buf.seek(0)
# buf.read() → bytes prontos para enviar
```

Imagens raster (bitmaps), como .png

```
fig.savefig("fig.png", dpi=300, bbox_inches="tight",
    transparent=False, facecolor="white")
```

- Resolução (dpi): 72 (web), 150 (slides), 300-600 (impressão)
  - Padrão do matplotlib: 100
- transparent=True — fundo sem cor (para sobrepor em slides ou sites)
- facecolor="white" — força fundo branco quando transparent=False
- Suporta canal alpha (RGBA)

Imagens vetoriais com SVG

```
fig.savefig("fig.svg", bbox_inches="tight", transparent=True,
  metadata={"Title": "Meu gráfico", "Creator": "matplotlib", },)
```

- Resolução “infinita”
- Arquivo de texto — versionável com Git
- Editável pós-exportação (cores, textos)
  - svg.fonttype="none" (padrão) exige que a fonte esteja instalada
  - Alternativa: plt.rcParams["svg.fonttype"] = "path"
- Imagens raster embutidas no SVG aumentam o tamanho do arquivo
- Gráficos com muitos pontos (> 50 k) geram SVGs lentos

Imagens vetoriais diretamente em PDF

```
fig.savefig("fig.pdf", bbox_inches="tight",
    metadata={"Title": "Figura 1", "Author": "Nome", },)
```

```
from matplotlib.backends.backend_pdf  import PdfPages

with PdfPages("relatorio.pdf") as pdf:
    pdf.savefig(fig1)
    pdf.savefig(fig2)
    pdf.savefig(fig3)
```

Exercício (em aula)

Vamos construir um painel 2×2 com dados fictícios de uma loja – uma espécie de *dashboard.*

O painel deverá ter quatro gráficos:

- Vendas mensais com meta (linha + área)
- Distribuição de ticket médio (histograma)
- Receita por categoria (waffle)
- Evolução mensal de 3 produtos (linhas)

```
# Geração de dados sintéticos
meses = ["Jan", "Fev", "Mar", "Abr", "Mai", "Jun"]
vendas   = [42, 38, 51, 47, 60, 55]
meta     = [45] * 6
tickets  = np.random.lognormal(4.5, 0.4, 500)

categorias = ["Eletrônicos", "Moda", "Casa",
"Esportes"]
receitas   = [185, 120, 95, 60]

np.random.seed(7)
prod_a = np.cumsum(np.random.randn(6)) + 20
prod_b = np.cumsum(np.random.randn(6)) + 15
prod_c = np.cumsum(np.random.randn(6)) + 10
```

Dica: subplots

```
fig, axes = plt.subplots(2, 2, figsize=(12, 8))
fig.suptitle("Dashboard de Vendas – 1º Semestre")

ax1 = axes[0, 0]
#ax1. (...)
ax2 = axes[0, 1]
#ax2. (...)
ax3 = axes[1, 0]
#ax3. (...)
ax4 = axes[1, 1]
#ax4. (...)

plt.tight_layout()
```

Projeto 8+

Vamos construir gráficos relevantes sobre o projeto anterior (microdados do ENEM). Selecione recortes dos dados que possam exemplificar os tipos de gráficos que trabalhamos (pelo menos 4 diferentes). Por exemplo:

- Histograma das diferentes provas para um dado conjunto populacional
- Gráficos de barra com os desempenhos de algumas cidades de uma dada unidade federada
- Gráfico *waffle* mostrando os recortes em uma dada capital dos desempenhos médios